Light

Programming — Testing & Robustness

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Types of errors
2. Testing strategies
3. Defensive design
Worked examples
Quick reference
Typical pitfalls

Edexcel GCSE Computer Science — **Programming: Testing and Robustness (1CP2/02 Topic 6: Problem Solving with Programming)**. This topic covers how to identify and fix errors in programs, how to test programs systematically using different types of test data, and how to make programs robust through validation and defensive design. All code examples use Python 3 following the Edexcel Programming Language Subset (PLS). Paper 2 requires you to test and refine programs. You need to understand the difference between **syntax, logic, and runtime errors**, select appropriate **test data**, and write code that handles unexpected inputs gracefully. Expect questions asking you to identify errors, suggest test data, and add validation to programs.

Why testing and robustness matter

No programmer writes perfect code the first time. Testing and debugging are essential skills that help you create reliable programs:

- **Finding errors:** Testing reveals bugs before users encounter them — saving time and preventing problems
- **Different error types:** Understanding syntax, logic, and runtime errors helps you fix them faster

- **Systematic testing:** Using normal, boundary, and erroneous data ensures thorough testing
- **Robust programs:** Validation prevents crashes from unexpected input — programs should fail gracefully
- **Professional practice:** Real-world software goes through extensive testing before release
- **Exam relevance:** Paper 2 explicitly requires you to "test and refine programs" — testing questions appear in every exam

What you must know

This checklist covers the key testing and robustness concepts from Edexcel 1CP2 Topic 6. You must be able to identify errors, design tests, and write defensive code.

- **Identify** syntax errors — mistakes in the code structure that prevent the program from running
- **Identify** logic errors — the program runs but produces incorrect results
- **Identify** runtime errors — errors that occur during program execution, causing it to crash
- **Select** appropriate test data: normal, boundary, and erroneous (invalid) data
- **Create** test plans with expected outcomes for each test
- **Use** iterative testing — testing during development, after each change
- **Use** final/terminal testing — testing the complete program against requirements
- **Write** input validation to check data is reasonable before processing
- **Apply** defensive design — anticipating misuse and preventing crashes
- **Debug** programs using trace tables, print statements, and systematic approaches

> **Specification link:** This topic covers Edexcel 1CP2 Paper 2 Topic 6 — Problem solving with programming. The specification requires students to "test and refine programs" using appropriate test data and debugging techniques.

Toolkit

These are the key concepts and code patterns you need for testing and robustness questions.

Types of errors overview

Error type	When detected	What happens	Example
Syntax error	Before running (by interpreter)	Program won't start	Missing colon, bracket, or quote
Logic error	During testing (by programmer)	Program runs but gives wrong output	Using + instead of -
Runtime error	During execution	Program crashes mid-run	Division by zero, invalid input

Types of test data

Test data type	Purpose	Example (for age 18-65)
Normal data	Test typical, expected inputs	25, 40, 50
Boundary data	Test values at the edges of valid ranges	17, 18, 65, 66
Erroneous data	Test invalid inputs that should be rejected	-5, "abc", 200

Essential code patterns

Basic input validation

```
# Validate input is within range
age = int(input("Enter your age: "))
while age < 0 or age > 120:
    print("Invalid age. Please enter 0-120.")
    age = int(input("Enter your age: "))
print("Valid age:", age)
```

Checking for specific values

```
# Validate input is one of allowed options
choice = input("Enter Y or N: ")
while choice != "Y" and choice != "N":
    print("Invalid choice. Enter Y or N.")
    choice = input("Enter Y or N: ")
print("You chose:", choice)
```

Presence check (not empty)

```
# Check input is not empty
name = input("Enter your name: ")
while name == "":
    print("Name cannot be empty.")
    name = input("Enter your name: ")
print("Hello,", name)
```

Length check

```
# Check password length
password = input("Enter password (8+ characters): ")
while len(password) < 8:
    print("Password too short. Need 8+ characters.")
    password = input("Enter password (8+ characters): ")
print("Password accepted")
```

Type check (numeric input)

```
# Check input contains only digits
num_str = input("Enter a number: ")
while not num_str.isdigit():
    print("Invalid. Enter digits only.")
```

```
    num_str = input("Enter a number: ")
num = int(num_str)
print("You entered:", num)
```

1. Types of errors

Programs can contain three types of errors. Understanding these helps you identify and fix problems quickly.

1.1 Syntax errors

A **syntax error** occurs when code breaks the rules of the programming language. Python cannot understand the instruction and will not run the program.

Common causes:

- Missing colon after `if`, `for`, `while`, or `def`
- Missing or mismatched brackets `()`, `[]`, `{}`
- Missing or mismatched quotes `"` or `'`
- Incorrect indentation
- Misspelled keywords (e.g., `whille` instead of `while`)
- Using `=` instead of `==` in conditions

Example with syntax error:

```
if score >= 50
    print("Pass")
```

```
SyntaxError: expected ':'
```

Fixed:

```
if score >= 50:
    print("Pass")
```

> **Exam tip:** Syntax errors are found by the interpreter BEFORE the program runs. The error message usually tells you the line number and type of problem.

1.2 Logic errors

A **logic error** occurs when the program runs without crashing but produces the wrong output. The code is syntactically correct but doesn't do what was intended.

Common causes:

- Using the wrong operator (+ instead of -)
- Wrong comparison (> instead of >=)
- Incorrect loop range (off-by-one errors)
- Conditions in wrong order
- Variables initialised incorrectly
- Updating wrong variable

Example with logic error:

```
# Intended to calculate average of 3 numbers
total = num1 + num2 + num3
```

```
average = total / 2  # Should be / 3
```

This runs without error but gives wrong results.

Fixed:

```
total = num1 + num2 + num3
average = total / 3
```

> **Exam tip:** Logic errors are the hardest to find because the program runs. You need to test with known inputs and check the outputs match expectations.

1.3 Runtime errors

A **runtime error** occurs when the program is running and encounters something it cannot handle. The program starts but then crashes.

Common causes:

- Division by zero
- Converting non-numeric string to int (e.g., `int("abc")`)
- Accessing a list index that doesn't exist
- Using a variable that hasn't been defined
- Opening a file that doesn't exist

Example — Division by zero:

```
num1 = 10
num2 = 0
result = num1 / num2
```

```
ZeroDivisionError: division by zero
```

Example — Invalid type conversion:

```
age = int(input("Enter age: "))
# User enters "twenty"
```

```
ValueError: invalid literal for int()
```

Example — Index out of range:

```
scores = [85, 90, 78]
print(scores[5])  # Only indices 0, 1, 2 exist
```

```
IndexError: list index out of range
```

> **Exam tip:** Runtime errors happen DURING execution. They're often caused by

unexpected user input — which is why validation is so important.

1.4 Comparing error types

Feature	Syntax error	Logic error	Runtime error
Program runs?	No — won't start	Yes — runs completely	Partially — crashes mid-run
Error message?	Yes — before running	No — no error shown	Yes — when crash occurs
How to find?	Interpreter reports it	Testing with known data	Happens during use
Difficulty to fix	Usually easy	Can be hard	Medium

Quick decision guide: Which error type?

```
Does the program START running?
├── NO → SYNTAX ERROR (code breaks language rules)
└── YES → Does the program CRASH during execution?
        ├── YES → RUNTIME ERROR (e.g., division by zero)
        └── NO → Does it give CORRECT output?
                ├── YES → No error!
                └── NO → LOGIC ERROR (wrong algorithm)
```

Worked Example 1 — Identifying error types

Question: Identify the type of error in each code snippet. [3 marks]

(a)

```
for i in range(10)
    print(i)
```

(b)

```
numbers = [1, 2, 3]
print(numbers[3])
```

(c)

```
celsius = 25
fahrenheit = celsius * 9 / 5 - 32  # Should be + 32
```

Model answer:

(a) **Syntax error (1)** — missing colon after `range(10)`
(b) **Runtime error (1)** — index 3 doesn't exist (valid indices are 0, 1, 2)
(c) **Logic error (1)** — program runs but formula is wrong (subtracts instead of adds 32)

Exam tip: Ask yourself: Does the program run? If not, it's syntax. If it runs but crashes, it's runtime. If it runs but gives wrong answers, it's logic.

Worked Example 2 — Fixing syntax errors

Question: The following code contains two syntax errors. Identify and correct them. [4 marks]

```
name = input("Enter name: )
if name == "Admin"
    print("Welcome, administrator")
```

Model answer:
Error 1: Line 1 — missing closing quote before the bracket **(1)**
Fix 1: `name = input("Enter name: ")` **(1)**
Error 2: Line 2 — missing colon after the if condition **(1)**
Fix 2: `if name == "Admin":` **(1)**
Corrected code:

```
name = input("Enter name: ")
if name == "Admin":
    print("Welcome, administrator")
```

Exam tip: Check for missing colons, quotes, and brackets — these are the most common syntax errors.

Worked Example 3 — Fixing logic errors

Question: The program below should count down from 5 to 1, but it doesn't work correctly. Identify and fix the logic error. [3 marks]

```
for i in range(5):
    print(i)
```

Model answer:
Error: `range(5)` produces 0, 1, 2, 3, 4 — not 5 to 1 **(1)**
Fix: Use `range(5, 0, -1)` to count from 5 down to 1 **(1)**

```
for i in range(5, 0, -1):
    print(i)
```

Output: 5, 4, 3, 2, 1 **(1)**

Exam tip: Logic errors need you to understand what the code SHOULD do, then compare with what it ACTUALLY does.

Worked Example 4 — Preventing runtime errors

Question: The program below crashes if the user enters a non-numeric value. Add validation to prevent this runtime error. [3 marks]

```
age = int(input("Enter your age: "))
print("Next year you will be", age + 1)
```

Model answer:

```
age_input = input("Enter your age: ")
while not age_input.isdigit():
    print("Invalid input. Enter a number.")
    age_input = input("Enter your age: ")
age = int(age_input)
print("Next year you will be", age + 1)
```

(1) Get input as string first
(1) Use `isdigit()` to check if input is numeric
(1) Loop until valid input, then convert to integer

Exam tip: The `isdigit()` method returns True if all characters are digits. It's useful for validating numeric input.

2. Testing strategies

Systematic testing uses carefully chosen test data to check that a program works correctly in all situations.

2.1 Normal data

Normal data (also called valid or typical data) represents inputs that are expected during normal use. These should be accepted and processed correctly.

Example: For a program accepting ages 18-65:

- 25 — typical adult age
- 40 — middle of range
- 55 — older but valid

Normal data tests that the program works for typical use cases.

2.2 Boundary data

Boundary data (also called extreme data) tests values at the edges of valid ranges. Many bugs occur at boundaries.

Example: For ages 18-65:

- 17 — just below minimum (should be rejected)
- 18 — exactly at minimum (should be accepted)
- 65 — exactly at maximum (should be accepted)
- 66 — just above maximum (should be rejected)

> **Exam tip:** Always test BOTH sides of a boundary. For range 18-65, test 17, 18, 65, AND 66.

2.3 Erroneous data

Erroneous data (also called invalid data) represents inputs that should be rejected. The program should handle these gracefully without crashing.

Example: For ages 18-65:

- -5 — negative age (impossible)
- "abc" — text instead of number
- 200 — unrealistic age
- "" — empty input

Erroneous data tests that validation works and the program doesn't crash.

2.4 Creating a test plan

A **test plan** documents what tests will be performed and what results are expected.

Test	Data type	Input	Expected output
1	Normal	25	Accepted
2	Boundary	18	Accepted
3	Boundary	17	Rejected
4	Erroneous	"abc"	Error message

2.5 Iterative vs final testing

Iterative testing

Testing performed **during development**, after each small change or new feature.

Benefits:

- Errors found quickly after being introduced
- Easier to identify which change caused a bug
- Problems fixed while code is fresh in mind

Example: After writing an input validation function, test it immediately with normal, boundary, and erroneous data before moving on.

Final testing (terminal testing)

Testing performed when the program is **complete**, checking it meets all requirements.

Benefits:

- Tests the complete system working together
- Checks all requirements are met
- May find issues from component interactions

Example: Testing the entire program against the original specification, using a comprehensive test plan.

2.6 Debugging techniques

Debugging is the process of finding and fixing errors. Several techniques can help:

Print statements

Add `print()` statements to show variable values at key points:

```
total = 0
for i in range(5):
    num = int(input("Enter number: "))
    print("DEBUG: num =", num)  # Debug line
    total = total + num
    print("DEBUG: total =", total)  # Debug line
print("Final total:", total)
```

Remove debug lines after fixing the error.

Trace tables

Track variable values through each step of execution:

Step	i	num	total
Start	–	–	0
1	0	5	5
2	1	3	8
3	2	7	15

Worked Example 5 — Selecting test data

Question: A program accepts exam scores between 0 and 100. Suggest appropriate test data for each category. [6 marks]

Model answer:

Normal data:

- 50 — typical score in middle of range **(1)**
- 75 — another typical valid score **(1)**

Boundary data:

- 0 — minimum valid score **(1)**

- 100 — maximum valid score **(1)**

Erroneous data:

- -1 — below minimum (invalid) **(1)**
- 101 — above maximum (invalid) **(1)**

Exam tip: For full marks, give specific values, not just descriptions. Say "50" not "a typical score".

Worked Example 6 – Creating a test plan

Question: A password must be at least 8 characters and no more than 20 characters. Complete the test plan. [6 marks]

Model answer:

Test	Type	Test data	Expected result
1	Normal	"password123" (11 chars)	Accepted **(1)**
2	Boundary (min)	"abcdefgh" (8 chars)	Accepted **(1)**
3	Boundary (min-1)	"abcdefg" (7 chars)	Rejected **(1)**
4	Boundary (max)	"abcdefghij1234567890" (20 chars)	Accepted **(1)**
5	Boundary (max+1)	"abcdefghij12345678901" (21 chars)	Rejected **(1)**
6	Erroneous	"" (empty)	Rejected **(1)**

Exam tip: For ranges with both minimum AND maximum, test boundaries at BOTH ends: min-1, min, max, and max+1.

Worked Example 7 – Explaining test data types

Question: Explain what is meant by boundary test data and why it is important. [3 marks]

Model answer:

Boundary test data tests values at the edges/limits of valid ranges **(1)**.

For example, if valid values are 1-100, boundary data would be 0, 1, 100, and 101 **(1)**.
It is important because many programming errors occur at boundaries, such as using > instead of >= **(1)**.

Exam tip: Always give an example when explaining test data types — it demonstrates understanding.

Worked Example 8 – Using a trace table to debug

Question: The program below should calculate the sum of numbers 1 to 5, but it gives the wrong answer. Use a trace table to identify the error. [4 marks]

```
total = 0
for i in range(5):
    total = total + i
print(total)
```

Model answer:
Trace table: **(2)**

i	total
0	0
1	1
2	3
3	6
4	10

Output is 10, but sum of 1+2+3+4+5 should be 15 **(1)**.
Error: `range(5)` gives 0-4, not 1-5. Fix: use `range(1, 6)` **(1)**.

Exam tip: Trace tables help you see exactly what values variables have at each step — making logic errors visible.

3. Defensive design

Defensive design means writing programs that anticipate problems and handle them gracefully, rather than crashing.

3.1 Input validation

Input validation checks that data entered by users is sensible and safe before the program uses it.

Types of validation checks:

- **Range check:** Is the value within acceptable limits?
- **Type check:** Is the data the correct type (number, text)?
- **Length check:** Is the data the right length?
- **Presence check:** Has data been entered (not empty)?
- **Format check:** Does the data match a required pattern?

> **Exam tip:** Validation checks if data is REASONABLE. It can't check if it's CORRECT — someone could enter a valid but wrong date of birth.

3.2 Why validation matters

Without validation, programs can:

- **Crash:** Converting "abc" to an integer causes a runtime error
- **Produce wrong results:** Negative quantities in a shopping cart
- **Be vulnerable:** Malicious input could cause security issues
- **Corrupt data:** Invalid data stored in files or databases

Validation example:

```
# Range check for percentage (0-100)
mark = int(input("Enter mark: "))
while mark < 0 or mark > 100:
    print("Error: must be 0-100")
    mark = int(input("Enter mark: "))
```

3.3 Validation code patterns

Range check

```
# Accept ages 16 to 65 only
age = int(input("Enter age: "))
while age < 16 or age > 65:
    print("Age must be between 16 and 65")
    age = int(input("Enter age: "))
print("Age accepted:", age)
```

Type check with isdigit()

```
# Check input is a whole number
user_input = input("Enter a number: ")
while not user_input.isdigit():
    print("Please enter digits only")
    user_input = input("Enter a number: ")
number = int(user_input)
print("You entered:", number)
```

Presence check

```
# Ensure name is not empty
name = input("Enter your name: ")
while name == "":
    print("Name cannot be blank")
```

```
    name = input("Enter your name: ")
print("Hello,", name)
```

Length check

```
# PIN must be exactly 4 digits
pin = input("Enter 4-digit PIN: ")
while len(pin) != 4 or not pin.isdigit():
    print("PIN must be exactly 4 digits")
    pin = input("Enter 4-digit PIN: ")
print("PIN accepted")
```

Combined validation

```
# Multiple checks: age must be number, 18-100
age_input = input("Enter age: ")
while not age_input.isdigit() or int(age_input) < 18 or int(age_input) > 100:
    print("Enter a valid age (18-100)")
    age_input = input("Enter age: ")
age = int(age_input)
print("Age accepted:", age)
```

3.4 Authentication

Authentication confirms a user's identity before allowing access. Simple authentication uses usernames and passwords.

```
# Simple password check
CORRECT_PASSWORD = "secret123"
MAX_ATTEMPTS = 3

attempts = 0
logged_in = False

while attempts < MAX_ATTEMPTS and not logged_in:
    password = input("Enter password: ")
    if password == CORRECT_PASSWORD:
        logged_in = True
        print("Access granted")
    else:
        attempts = attempts + 1
        remaining = MAX_ATTEMPTS - attempts
        print("Incorrect. Attempts remaining:", remaining)

if not logged_in:
    print("Account locked")
```

Exam tip: Good authentication limits the number of attempts to prevent guessing attacks.

Worked Example 9 — Writing a range check

Question: Write Python code to ask for a month number (1-12) and keep asking until valid input is entered. [4 marks]

Model answer:

```
month = int(input("Enter month (1-12): "))
while month < 1 or month > 12:
    print("Invalid month")
    month = int(input("Enter month (1-12): "))
print("Month:", month)
```

(1) Get initial input with appropriate prompt
(1) While loop condition checks if outside valid range
(1) Error message displayed for invalid input
(1) Re-prompt inside the loop

Exam tip: For "outside range" validation, use OR: `month < 1 or month > 12`. The value is invalid if it's below minimum OR above maximum.

Worked Example 10 — Writing a length check

Question: A username must be between 3 and 15 characters. Write validation code. [4 marks]

Model answer:

```
username = input("Enter username (3-15 chars): ")
while len(username) < 3 or len(username) > 15:
    print("Username must be 3-15 characters")
    username = input("Enter username (3-15 chars): ")
print("Username accepted:", username)
```

(1) Initial input
(1) Use `len()` to check string length
(1) Correct condition for outside valid range
(1) Re-prompt for input in loop

Exam tip: `len()` returns the number of characters in a string. It's essential for length validation.

Worked Example 11 — Explaining validation types

Question: For each validation type, give an example of when it would be used. [4 marks]

Model answer:
Range check: Checking an exam mark is between 0 and 100 **(1)**

Type check: Checking that input for a calculator is a number, not text **(1)**
Length check: Checking a password is at least 8 characters long **(1)**
Presence check: Checking a name field is not left empty on a form **(1)**

Exam tip: Give realistic examples — think about real programs and what validation they would need.

Worked Example 12 – Complete validation program

Question: Write a program that asks for a quantity of items to order. The quantity must be a whole number between 1 and 99. Include appropriate validation. [6 marks]

Model answer:

```
# Get quantity with validation
qty_input = input("Enter quantity (1-99): ")

# Type check - ensure it's a number
while not qty_input.isdigit():
    print("Error: enter a whole number")
    qty_input = input("Enter quantity (1-99): ")

qty = int(qty_input)

# Range check - ensure it's in valid range
while qty < 1 or qty > 99:
    print("Error: quantity must be 1-99")
    qty = int(input("Enter quantity (1-99): "))

print("You ordered", qty, "items")
```

(1) Initial input as string
(1) Type check using `isdigit()`
(1) Appropriate error message for type error
(1) Range check for 1-99
(1) Appropriate error message for range error
(1) Final output confirming valid quantity

Exam tip: For multi-mark validation questions, consider BOTH type validation (is it a number?) AND range validation (is it in the allowed range?).

Worked examples summary

This topic includes 12 worked examples covering the key testing and robustness skills you need for Paper 2.

Example	Skill tested	Topic
1	Identifying error types	Errors
2	Fixing syntax errors	Errors
3	Fixing logic errors	Errors
4	Preventing runtime errors	Errors
5	Selecting test data	Testing
6	Creating a test plan	Testing
7	Explaining test data types	Testing
8	Using trace tables to debug	Testing
9	Writing a range check	Validation
10	Writing a length check	Validation
11	Explaining validation types	Validation
12	Complete validation program	Validation

Quick reference

Key terms

Term	Definition
Syntax error	An error in the code structure that prevents the program from running — detected by the interpreter before execution
Logic error	An error where the program runs but produces incorrect results because the code doesn't do what was intended
Runtime error	An error that occurs during program execution, causing it to crash
Normal test data	Typical, expected inputs that should be accepted and processed correctly
Boundary test data	Values at the edges of valid ranges — testing both sides of limits
Erroneous test data	Invalid inputs that should be rejected — testing that validation works
Iterative testing	Testing performed during development, after each change or new feature

Term	Definition
Final testing	Testing performed when the program is complete, checking all requirements are met
Input validation	Checking that user input is reasonable and safe before processing it
Defensive design	Writing programs that anticipate and handle problems gracefully without crashing
Debugging	The process of finding and fixing errors in a program
Trace table	A table tracking variable values through each step of program execution — used to find logic errors

Validation code patterns

Validation type	Python pattern
Range check	`while value < min or value > max:`
Type check (digits)	`while not input_str.isdigit():`
Presence check	`while input_str == "":`
Length check (min)	`while len(input_str) < min_length:`
Length check (exact)	`while len(input_str) != required_length:`
Specific values	`while value != "Y" and value != "N":`

Common error messages

Error message	Type	Cause
`SyntaxError: expected ':'`	Syntax	Missing colon after if/for/while/def
`SyntaxError: EOL while scanning string`	Syntax	Missing closing quote on string
`IndentationError`	Syntax	Incorrect or inconsistent indentation
`NameError: name 'x' is not defined`	Runtime	Using a variable before assigning it
`TypeError`	Runtime	Operation on incompatible types
`ValueError`	Runtime	Invalid value (e.g., int("abc"))

Error message	Type	Cause
`ZeroDivisionError`	Runtime	Division by zero
`IndexError`	Runtime	List index out of range

Typical pitfalls

These are common mistakes students make in testing and robustness questions. Avoiding these errors will help you gain marks in Paper 2.

Pitfall 1 — Confusing error types.
Wrong: Calling a logic error a "runtime error" because it gives wrong results.
Right: Logic errors don't crash — the program runs but gives wrong output. Runtime errors cause crashes.

Pitfall 2 — Only testing normal data.
Wrong: Testing only with typical values like 25, 50.
Right: Test ALL three types: normal, boundary, AND erroneous data.

Pitfall 3 — Missing boundary values.
Wrong: For range 1-100, only testing 0 and 101.
Right: Test BOTH sides: 0, 1, 100, AND 101. Include values just inside AND just outside.

Pitfall 4 — Using AND instead of OR for range validation.
Wrong: `while num < 1 and num > 100` (impossible condition!)
Right: `while num < 1 or num > 100` — invalid if below minimum OR above maximum.

Pitfall 5 — Not including expected results in test plans.
Wrong: Test plan that only lists test data.
Right: Include what SHOULD happen for each test — the expected output or result.

Pitfall 6 — Forgetting to re-prompt in validation loops.
Wrong: While loop that checks but never asks for new input.
Right: Include `input()` inside the loop to get new data.

Pitfall 7 — Converting input before checking type.
Wrong: `num = int(input(...))` then checking if valid — crashes on non-numeric input.
Right: Get as string first, use `isdigit()`, then convert.

Pitfall 8 — Vague test data descriptions.
Wrong: "A normal value" or "An invalid input".
Right: Give specific values: "50" or "-5" or "abc".

Pitfall 9 — Confusing validation and verification.
Wrong: Saying validation checks if data is correct.
Right: Validation checks if data is REASONABLE. Verification checks if data is CORRECT (e.g., re-typing email).

Pitfall 10 — Not explaining WHY boundary testing is important.
Wrong: Just saying "to test the limits".
Right: Explain that errors often occur at boundaries due to mistakes like > instead of >=.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.